AutoBench Service — Developer Guide

Last modified: 2026-09-07T04:03:00Z

> Hand-maintained, unlike the generated `results/12run-*.md` files which stamp themselves. Bump the line above when you edit this guide.

A task-oriented guide to driving the AutoBench Service over its RESTful API. Every `curl` below is derived from the flows we exercised end-to-end (gsm8k single-turn, tau2 multi-turn) on the `ykt3` and `kind-rossoctl` clusters.

- Reference the machine-readable contract in `openapi.json` / `openapi.yaml`.
- Design rationale (what the Service can and cannot enact) lives in `SERVICE_DESIGN_DECISIONS.md` and `KUBECTL_DEPENDENCY_INVENTORY.md`.

In a hurry? §2 gets you a token, §6.0 is one benchmark run start to finish as copy-pasteable `curl`, and §6.1 is the same thing as a single command.

Contents

---

1. Mental model (read this first)

The Service is **pure-Python and HTTP-only**. It never calls a cluster API (no kubectl, no kubeconfig). It talks to the Rossoctl/kagenti backend over HTTPS and to Keycloak for tokens. Two consequences shape everything below:

1. **Two independent tokens.**
   - The **caller JWT** you present in `Authorization: Bearer …` is used *only* for attribution (`preferred_username`) and routing (`iss` selects which instance config applies). It is **never forwarded** upstream.
   - For each request the Service mints its **own** ROPC token (the instance's `benchmarker` credential, password grant) and presents *that* to Rossoctl.
2. **The Service enacts only what HTTP allows.** It deploys workloads (agents/tools), runs benchmarks, reads reports, and exports to S3. It **cannot** create cluster Secrets or overlay per-agent ConfigMaps — those are provisioned out-of-band, and the Service reports on them (see §3 onboarding and the `424` / `422` responses).

Endpoint map

Group	Endpoints	Auth
Identity	`GET /hello`, `GET /healthz` (public, unschematized)	caller JWT
Config	`GET /config`, `PUT /config`	**benchmarker only**
Discovery	`GET /namespaces`, `GET /benchmarks`, `GET /benchmarks/{name}`	caller JWT
Raw workloads	`GET/POST /agents`, `GET/DELETE /agents/{ns}/{name}`, same for `/tools`	caller JWT
Benchmark deploy	`POST /benchmarks/{name}/deploy`, `DELETE /benchmarks/{name}/de`	caller JWT

Group	Endpoints	Auth
	ploy, GET /benchmarks/{name}/status	
Run lifecycle	POST /benchmarks/{name}/runs, GET …/runs, GET …/runs/{run_id}	caller JWT
Reporting	GET …/runs/{run_id}/report, GET /benchmarks/{name}/report	caller JWT

2. Getting a caller token

The caller JWT is a normal Keycloak token from the instance's realm. Obtain it via the password grant (the same Direct-Access-Grants flow the dev/test users use). The realm and token endpoint are derived from the instance `iss` (`<iss-origin>/realms/<realm>/protocol/openid-connect/token`).

```
# --- Environment (pick the block for your target cluster) ---

# Option A — ykt3 (OpenShift; realm/client "kagenti"), the cluster the e2e
runs were validated on:
export SVC="https://autobench.apps.ykt3.example.com"     # AutoBench Service
base URL
export KC="https://keycloak.apps.ykt3.example.com"          # Keycloak base
(iss origin)
export REALM="kagenti"                                       # realm from the
iss
export KC_CLIENT="kagenti"                                   # public client
with Direct Access Grants

# Option B — kind-rossoctl (local; realm/client "rossoctl"):
#   The instance's iss is http://keycloak.localtest.me:8080/realms/rossoctl,
but in-cluster the
#   Service reaches Keycloak via keycloak_backchannel_url. You still fetch
YOUR token from the
#   externally reachable route below (localtest.me resolves to 127.0.0.1).
# export SVC="http://autobench.localtest.me:8080"
# export KC="http://keycloak.localtest.me:8080"
# export REALM="rossoctl"
# export KC_CLIENT="rossoctl"

# Attribution matters: config endpoints require preferred_username ==
"benchmarker".
# Any valid realm user works for deploy/run/report.
# Never hardcode the password — supply it out-of-band (env, secret manager,
```

```
or an
# interactive prompt). e.g.: read -rs -p "benchmarker password: " KC_PASS;
export KC_PASS
export KC_USER="benchmarker"
export KC_PASS="${KC_PASS:?set KC_PASS in your environment; do not commit
it}"

export TOKEN=$(curl -s "$KC/realms/$REALM/protocol/openid-connect/token" \
  -d grant_type=password \
  -d client_id="$KC_CLIENT" \
  -d username="$KC_USER" \
  -d password="$KC_PASS" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)
["access_token"])')

# Sanity check: the Service echoes the validated claims + which instance you
routed to.
curl -s "$SVC/hello" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

`GET /hello` returns `{iss, preferred_username, rossoctl_base_url, claims}`. If you get `401`, the token is missing/invalid; `403 issuer not in scope` means no instance file matches the token's `iss` (see §3.1).

> **Why the kind block differs:** in-cluster the public `iss` host is unreachable, so the instance file sets `keycloak_backchannel_url` and the Service dials *that* for JWKS/ROPC. This only affects the Service's own dialing — your caller token still comes from the externally reachable Keycloak route (Option B above).

---

3. Onboarding a benchmark (one-time, per cluster/instance)

These steps are **out-of-band** — the Service verifies them but cannot perform them.

3.1 Instance config file (`instances/<encoded-iss-host>.json`)

One file per instance keys the whole thing to the `iss`. Loaded at startup from `settings.instances_dir`. Shape:

```
{
  "iss": "https://keycloak.apps.ykt3.example.com/realms/kagenti",
  "keycloak_backchannel_url": null,
  "rossoctl_base_url": "https://kagenti-backend.kagenti-
system.svc.cluster.local:8000",
  "service_credential": {
    "client_id": "kagenti",
    "client_secret": "",
    "username": "benchmarker",
    "password": "<benchmarker-password>"
  },
  "mlflow": { "tracking_url": null },
```

```
  "s3": { "bucket": null },
  "mcp_endpoint_template": null,
  "agent_endpoint_template": null,
  "workload_otel": null
}
```

- `service_credential` is the ROPC identity the Service presents to Rossoctl (must exist in the realm with `firstName`/`lastName` set, or Keycloak 400s with "Account is not fully set up").
- `mcp_endpoint_template` / `agent_endpoint_template` are only needed when the workloads live on a **different** cluster reachable via external routes (use `{service}`/`{namespace}` placeholders). Leave `null` for co-located in-cluster workloads.
- `mlflow` / `s3` can be seeded here or set later via `PUT /config` (§4).

3.2 Infrastructure resources (baked into the benchmark definitions)

The Service sends CPU/memory requests+limits in the create body, so no post-create patch is needed. Current values (from `benchmarks/registry.py`):

Workload	requests	limits
MCP tool	`500m` CPU / `512Mi`	`4` CPU / `4Gi`
A2A agent	`500m` CPU / `512Mi`	`4` CPU / `2Gi`

3.3 Workload secrets (provisioned out-of-band as cluster Secrets)

The Service has **no Secrets API**. It references these by name in the deploy body; if a Secret is missing the workload never becomes Ready, and the run precheck returns `424` naming exactly what to provision.

Benchmark	Secret (`name` → `key`)	Consumed by	Purpose
gsm8k	`hf-secret` → `hf-token`	tool	HuggingFace dataset access
gsm8k / tau2 / appworld	`openai-secret` → `apikey`	agent (+ tau2/appworld tool)	LiteLLM API key

Provision them on the workload cluster before deploying, e.g.:

```
kubectl -n team1 create secret generic hf-secret --from-literal=hf-
token="$HF_TOKEN"
kubectl -n team1 create secret generic openai-secret --from-
literal=apikey="$LITELLM_KEY"
```

> `openai-secret` **must exist before you run the Service — provision it out-of-band.** On clusters where the rossoctl platform chart owns the agent namespaces (e.g. `team1`), the chart only manages `openai-secret` when its Helm value `secrets.openaiApiKey` is set. Leave that value **empty** and create the Secret yourself (command above / a secret manager / External Secrets), so the credential lives outside Helm values and a later `helm upgrade` **cannot** overwrite it with an empty key. When the value is empty the chart skips the Secret entirely and its install NOTES print a

`WARNING: secrets.openaiApiKey is NOT set` reminder. If you instead let the chart template the key, every `helm upgrade` re-applies whatever is in the release values — an empty value there silently zeroes the key mid-flight and every LLM call fails until the Secret is restored and the workload pods are `rollout restart`ed (pods read `apikey` via `secretKeyRef` only at startup).

3.4 MLflow + OTEL collector (optional, for reports)

Reporting is fail-soft: if MLflow client-creds aren't configured, runs still succeed and export to S3, but `report.ndjson` is empty and the report endpoints return `409`. To enable, provision an OTEL collector (forwards agent spans to MLflow) out-of-band and set the MLflow read creds via `PUT /config`.

---

4. Instance-specific Service config (`/config`)

`GET/PUT /config` are **benchmarker-only** (`preferred_username == "benchmarker"`; else `403`). They set only what the Service itself enacts — MLflow (read side) and S3. Attempting to set a workload credential is rejected with `422` (`extra="forbid"`). Secrets are redacted in responses.

```
# View effective config for your instance (file defaults + any overrides),
secrets redacted.
curl -s "$SVC/config" -H "Authorization: Bearer $TOKEN" | python3 -m
json.tool

# Point the Service's result sink at an S3 bucket + wire up MLflow read
creds.
curl -s -X PUT "$SVC/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{
        "s3": {
          "bucket": "rossoctl-benchmarking",
          "region": "us-east-1",
          "access_key_id": "AKIA...REDACTED",
          "secret_access_key": "REDACTED",
          "public_read": false
        },
        "mlflow": {
          "tracking_url": "https://mlflow.apps.ykt3.example.com",
          "client_id": "mlflow-client",
          "client_secret": "REDACTED",
          "token_url":
"https://keycloak.apps.ykt3.example.com/realms/kagenti/protocol/openid-
connect/token",
          "insecure_tls": false
        }
      }'
```

Rejected example (workload cred → `422`):

```
curl -s -o /dev/null -w '%{http_code}\n' -X PUT "$SVC/config" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"hf-secret": "..."}'          # -> 422
```

---

5. Benchmark lifecycle

The happy path is: **deploy → wait for Ready → run → poll → report → download from S3**.

Every block below is copy-pasteable once these four variables are exported (see §2 for how to get the token; §6.1 wraps this whole section in a single command if you would rather not paste):

```
export SVC="https://autobench-rossoctl-system.apps.ykt3.hcp.res.ibm.com"   # Service base URL
export BENCH=gsm8k          # gsm8k | tau2 | appworld
export SCOPE="namespace=team1&agent=tool_calling&experiment=default"
export TOKEN="…"            # from §2; never echo this
# Self-signed OpenShift route? add -k to every curl, or: export CURL_OPTS=-k
```

5.0 Discover what's available

```
curl -s "$SVC/benchmarks" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
curl -s "$SVC/benchmarks/gsm8k" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
curl -s "$SVC/namespaces" -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Three benchmarks are registered: `gsm8k` (single-turn), `tau2` (multi-turn, runs a server-side user-simulator LLM), `appworld` (registered; see §7).

5.1 Deploy (create the MCP tool + A2A agent)

`POST /benchmarks/{name}/deploy` creates both the shared MCP tool (`exgentic-mcp-<name>`) and the agent (`exgentic-a2a-<agent>-<name>[-<experiment>]`). Returns `201`.

```
curl -s -X POST "$SVC/benchmarks/gsm8k/deploy" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{
        "agent": "tool_calling",
        "namespace": "team1",
        "experiment": "default"
      }' | python3 -m json.tool
```

Request fields (`DeployBenchmarkRequest`):

Field	Default	Meaning
`agent`	`tool_calling`	agent flavor (only `tool_calling` today)
`model`	`null`	**deploy-time** model, baked into

Field	Default	Meaning
		agent env; `null` → benchmark default (`openai/Qwen3.6-35B-A3B`)
`namespace`	`team1`	target namespace
`experiment`	`default`	non-default suffixes the agent name so variants coexist
`authbridge_enabled`	`false`	inject the AuthBridge sidecar with the **cluster-default** pipeline (layer-2 knob)
`plugin_preset`	`null`	layer-3 preset (`auth-only`|`ibac-only`|`full`); forwarded to the backend as `pluginPreset` → `AgentRuntime.spec` → operator renders the per-agent pipeline. Requires `authbridge_enabled=true`
`plugins`	`null`	per-plugin policy overrides as `["NAME:POLICY"]` tokens (`POLICY` = `enforce`|`observe`|`off`); forwarded as `plugins`. Requires `authbridge_enabled=true`
`on_error`	`null`	chain-default policy (`enforce`|`observe`|`off`); forwarded as `onError`. Requires `authbridge_enabled=true`
`plugin_config_file`	`null`	**rejected with** `422` — a local filesystem path with no HTTP analog; use `plugin_preset`/`plugins`/`on_error` instead

Model swap (run #4 pattern): the deploy endpoint always (re)creates the *shared* tool, so re-deploying with a new model 409s on the existing tool. Deploy the model-swapped agent alone via `POST /agents` under a distinct experiment instead:

```
# Generate the agent body with build_agent_request semantics, then POST
/agents directly.
curl -s -X POST "$SVC/agents" \
```

```
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{
        "name": "exgentic-a2a-tool-calling-gsm8k-azmini",
        "namespace": "team1",
        "container_image": "ghcr.io/exgentic/exgentic-a2a-tool_calling",
        "image_tag": "latest",
        "env_vars": [
          {"name": "MCP_URL", "value": "http://exgentic-mcp-gsm8k-
mcp.team1.svc.cluster.local:8000/mcp"},
          {"name": "LLM_MODEL", "value": "openai/Azure/gpt-4o-mini"},
          {"name": "EXGENTIC_SET_AGENT_MODEL", "value": "openai/Azure/gpt-4o-
mini"},
          {"name": "OPENAI_API_KEY", "value_from": {"secret_key_ref":
{"name": "openai-secret", "key": "apikey"}}}
        ],
        "service_ports": [{"name": "http", "port": 8080, "target_port":
8000}]
      }' | python3 -m json.tool
```

5.2 Wait until Ready

```
curl -s "$SVC/benchmarks/gsm8k/status?
namespace=team1&agent=tool_calling&experiment=default" \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Returns `tool_ready` / `agent_ready` booleans plus raw `readyStatus`. Poll until both are `true`. If a workload stays not-Ready, the run precheck (§5.3) will name the missing Secret.

5.3 Submit a run

`POST /benchmarks/{name}/runs` runs a **cluster-API-free precheck** (tool+agent deployed and Ready) then returns `202` with a `run_id`. Prechecks:

- `409` — not deployed (`POST …/deploy` first).
- `424` — deployed but not Ready; the message names required Secret(s) (e.g. `hf-secret`).

```
export RUN=$(curl -s -X POST "$SVC/benchmarks/gsm8k/runs" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{
        "agent": "tool_calling",
        "namespace": "team1",
        "experiment": "default",
        "max_tasks": 10,
        "max_parallel_sessions": 1,
        "timeout_seconds": 600
      }' | python3 -c 'import sys,json; print(json.load(sys.stdin)
["run_id"])')
echo "run_id=$RUN"
```

Run fields (`RunRequest`) are all **run-time** knobs:

Field	Default	Notes
`max_tasks`	`1`	number of benchmark tasks to

Field	Default	Notes
		evaluate
`max_parallel_sessions`	`1`	concurrency
`timeout_seconds`	`300`	whole-run wall-clock ceiling; raise for large tau2 runs
`agent` / `namespace` / `experiment`	—	must match a deployed agent
`model`	`null`	**not** forwarded per-session; model is fixed at deploy time

Timeout tuning (learned e2e): tau2 with `max_tasks=20` `max_parallel_sessions=4` exceeded the 900s default and failed; re-running with `timeout_seconds=1800` succeeded (~915s).

5.4 Poll run status / list runs

```
# One run's full state (status, summary, per-task results, artifacts once
exported).
curl -s "$SVC/benchmarks/gsm8k/runs/$RUN" -H "Authorization: Bearer $TOKEN" |
python3 -m json.tool

# All runs for this benchmark scoped to your instance.
curl -s "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer $TOKEN" |
python3 -m json.tool
```

`status` moves `pending → running → succeeded|failed`. On success, `summary` carries `{total, succeeded, evaluated_pass, pass_rate, wall_seconds}`. A simple poll loop:

```
# Terminal statuses are succeeded | failed | error | cancelled — a loop that
waits only for
# succeeded/failed spins forever on the other two.
while :; do
  S=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H
"Authorization: Bearer $TOKEN" \
      | python3 -c 'import sys,json; print(json.load(sys.stdin)["status"])')
  echo "status=$S"
  case "$S" in succeeded|failed|error|cancelled) break ;; esac
  sleep 10
done

# The one-line verdict.
curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H "Authorization:
Bearer $TOKEN" > /tmp/run.json
python3 - <<'EOF'
import json
d = json.load(open("/tmp/run.json")); s = d["summary"]
print("%s  pass_rate=%s  %s/%s  wall=%.0fs" % (
    d["status"], s["pass_rate"], s["evaluated_pass"], s["total"],
s["wall_seconds"]))
EOF
```

5.5 Get results (report)

Two report views, both reading structured records from MLflow (return `409` if MLflow isn't configured for the instance — see §3.4):

```
# Per-run report: records filtered to this run's session ids, plus its S3
artifacts.
curl -s "$SVC/benchmarks/gsm8k/runs/$RUN/report" -H "Authorization: Bearer
$TOKEN" | python3 -m json.tool

# Experiment rollup: time-windowed aggregates across an experiment's runs.
curl -s "$SVC/benchmarks/gsm8k/report?experiment=default&window_h=3" \
  -H "Authorization: Bearer $TOKEN" | python3 -m json.tool
```

Per-run report is `{run_id, benchmark, experiment, trace_count, records[], artifacts[]}`; each record (`MLflowTraceRecord`) has per-session timing breakdown, LLM/tool latencies + token counts, infra CPU/mem, and the evaluation outcome.

5.6 Download output files from S3

When the instance has an S3 `bucket` set, each completed run is exported (fail-soft) and its objects appear in the run state under `artifacts[]` (and in the per-run report). Layout:

```
<prefix>/<preferred_username>/<encoded-iss>/<benchmark>/<run_id>/
    run.json                # the run summary (always written)
    report.ndjson           # one record per task (empty if MLflow
unconfigured)
    report.parquet          # analytics-friendly (only when there are
records)
    token_report.ndjson     # lean per-task token view, projected from the
records
    token_report.parquet
    span_report.ndjson      # one row per OTEL span per task, incl. the span
name
    span_report.parquet
    manifest.json           # self-describing index of the above (never lists
itself)
```

`span_report.*` is the evidence layer under the other two: `report`/`token_report` give per-task *counts*, `span_report` gives the spans those counts were derived from, so "did this task really make one model call, or did we lose a span?" is answerable from the artifacts alone. Its columns are a fixed whitelist (`mlflow_report.SPAN_ROW_KEYS`) — span attributes can carry prompts, and these objects are public-read, so nothing outside that list is published.

The S3 URL is fully determined — you can construct it

Objects are addressed as `<url-root>/<key>`, and the key is the layout above. For the bucket these runs use:

URL root	`https://rossoctl-`

Key	benchmarking.s3.us-east-1.amazonaws.com/<s3.prefix>/<preferred_username>/<encoded-iss-host>/<benchmark>/<run_id>/<artifact>

So a real, working URL — anonymously readable, no credentials, no AWS CLI:

```
https://rossoctl-benchmarking.s3.us-east-1.amazonaws.com/ykt3-to-ykt2/
benchmarker/keycloak-keycloak.apps.ykt2.hcp.res.ibm.com-realms-rossoctl/
gsm8k/20260902215628-79e0713a/report.ndjson
```

`s3.prefix` is per instance (`ykt3-to-ykt2/` on the OpenShift instance, `kind/` on KinD), and the issuer host is scheme-stripped with `./://` replaced by `-`. **The bucket is public and anonymously listable**, so treat everything exported as published.

```
# The authoritative list — name, format, size and the full URL, straight from
the run state.
curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H "Authorization:
Bearer $TOKEN" > /tmp/run.json
python3 - <<'EOF'
import json
for a in json.load(open("/tmp/run.json"))["artifacts"]:
    print("%-8s %9d  %s" % (a["format"], a["size_bytes"], a["url"]))
EOF

# Download every artifact by URL — plain curl, no AWS credentials needed.
PREFIX=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H
"Authorization: Bearer $TOKEN" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)
["artifacts_prefix"])')
mkdir -p "/tmp/autobench/$PREFIX" && cd "/tmp/autobench/$PREFIX"
for f in run.json report.ndjson report.parquet token_report.ndjson
token_report.parquet \
         span_report.ndjson span_report.parquet manifest.json; do
  curl -sS -O
"https://rossoctl-benchmarking.s3.us-east-1.amazonaws.com/$PREFIX/$f"
done && ls -la

# Or discover the set from the manifest, which indexes every object but
itself.
curl -s "https://rossoctl-benchmarking.s3.us-east-1.amazonaws.com/$PREFIX/
manifest.json" \
  | python3 -m json.tool

# Private bucket instead of public_read? Then use the AWS CLI with
credentials.
aws s3 cp "s3://rossoctl-benchmarking/$PREFIX/" ./ --recursive
```

5.7 Tear down

```
curl -s -X DELETE "$SVC/benchmarks/gsm8k/deploy?
namespace=team1&agent=tool_calling&experiment=default" \
```

```
  -H "Authorization: Bearer $TOKEN" -o /dev/null -w '%{http_code}\n'   # ->
204
```

Deletes the agent and the shared tool. For a model-swap variant deployed via `POST /agents`, delete it with `DELETE /agents/{namespace}/{name}` (the shared tool is left for other experiments).

---

6. End-to-end examples (validated flows)

What you need on the client side

Python version. The client side is deliberately undemanding: §6.0 needs only `curl` plus any `python3` for JSON formatting, and `autobench-cli` (§6.1) imports nothing beyond the standard library. Versions we have actually run:

Python	Where it was used	Result
3.12.12	recommended for a client venv — matches the workload images exactly	✅
3.12.14	the Service container itself (`FROM python:3.12-slim`)	✅
3.11.14	the repo's own venv; the declared floor (`requires-python = ">=3.11"`)	✅
3.14.3	the laptop that drove both 12-run matrices	✅

Anything **≥ 3.11** is fine. 3.12.12 is the tidiest choice because it is exactly what the exgentic agent and MCP images run, so your driver matches the workload interpreter — but nothing in the client depends on it, and the `curl` path in §6.0 is version-insensitive.

Do you have to clone the repo? For **§6.0, no** — it is `curl` and `python3` only, so a token and a Service URL are all you need. For **§6.1** you need the package that provides the `autobench-cli` entry point, but a manual clone is still optional: the repository is public, so install it straight from git.

```
# No clone. Client only: --no-deps, because the CLI is stdlib-only and pulls
in nothing else.
uv venv --python 3.12.12 && source .venv/bin/activate
uv pip install "rossoctl-autobench @
git+https://github.com/rossoctl/autobench" --no-deps

# Already have a clone? Point at the repo, NOT at `.` — `-e .` looks for
pyproject.toml in the
# CURRENT directory, so running it from a scratch/testbed folder fails with
# "does not appear to be a Python project".
```

```
uv pip install -e /path/to/autobench --no-deps        # -e: git pull updates
the CLI in place
```

Drop `--no-deps` only if you also want to run the Service or its test suite from that venv; it adds fastapi, boto3, pydantic and the MCP/A2A SDKs, none of which the client needs. Keep the venv outside a cloud-synced folder (Box/Dropbox/iCloud) — a venv is thousands of small files, and sync tools are slow and occasionally destructive with it.

Where the source ends up

`<venv>/bin/autobench-cli` is **not** the source — it is a generated shim that does `from autobench.cli import main`. Where the real `cli.py` lives depends on how you installed, which also decides whether a `git pull` reaches you:

Install	`cli.py` lives in	Picks up upstream changes?
`uv pip install "…@ git+https://…"`	`<venv>/lib/python3.X/site-packages/autobench/` (a **copy**, built from one commit)	No — pinned. Re-run with `--reinstall`
`uv pip install -e /path/to/autobench`	`/path/to/autobench/src/autobench/` (the clone; only a finder hook is installed)	Yes, immediately
`uv tool install --from /path/to/autobench autobench-service`	`$(uv tool dir)/rossoctl-autobench/lib/python3.X/site-packages/autobench/`	No — reinstall the tool
no install, `PYTHONPATH=<repo>/src python -m autobench.cli`	the clone itself	Yes, immediately

Never guess the path — ask the interpreter that is actually running it:

```
python -c "import autobench.cli as m; print(m.__file__)"
```

Note that every option installs the **whole distribution**, server modules included (`app.py`, `routes/`, `runner/`, `s3_export.py`), because they ship together. With `--no-deps` the client half still works while `import autobench.app` does not — the third-party dependencies were skipped, not the files. That the client keeps working is the point, and you can confirm it:

```
python -c "
import autobench.cli, sys
print([m for m in ('fastapi','httpx','boto3','pydantic') if m in sys.modules]
or 'no server deps loaded')"
```

6.0 Run #1 start to finish, on the ykt3 Service driving ykt2 workloads

Run #1 of the canonical matrix: gsm8k, 1 task, no gateway, no plugins. Every command below was executed exactly as written; the `run_id` and numbers are from that run. Paste the block, then the steps.

```
# ---- 0. environment (the cross-cluster instance: Service on ykt3, workloads
on ykt2/team1) ----
export SVC="https://autobench-rossoctl-system.apps.ykt3.hcp.res.ibm.com"
export KC="https://keycloak-keycloak.apps.ykt2.hcp.res.ibm.com"   # the
instance's iss origin
export REALM=rossoctl KC_CLIENT=rossoctl KC_USER=benchmarker
export BENCH=gsm8k
export SCOPE="namespace=team1&agent=tool_calling&experiment=default"
export CURL_OPTS=-k          # ykt3's route serves a self-signed cert
read -rs -p "benchmarker password: " KC_PASS; echo      # never put this in a
file you commit

# ---- 1. caller token ----
export TOKEN=$(curl -s $CURL_OPTS "$KC/realms/$REALM/protocol/openid-
connect/token" \
  -d grant_type=password -d client_id="$KC_CLIENT" \
  -d username="$KC_USER" -d password="$KC_PASS" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)
["access_token"])')
unset KC_PASS
curl -s $CURL_OPTS "$SVC/hello" -H "Authorization: Bearer $TOKEN" | python3
-m json.tool
#   -> {"iss": ".../realms/rossoctl", "preferred_username":
"benchmarker", ...}

# ---- 2. deploy the MCP tool + A2A agent (idempotent: delete first) ----
curl -s $CURL_OPTS -X DELETE "$SVC/benchmarks/$BENCH/deploy?$SCOPE" \
  -H "Authorization: Bearer $TOKEN" -o /dev/null -w 'delete -> %{http_code}
\n'   # 204 or 404
sleep 10
curl -s $CURL_OPTS -X POST "$SVC/benchmarks/$BENCH/deploy" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"agent":"tool_calling","namespace":"team1","experiment":"default"}' \
  -o /dev/null -w 'deploy -> %{http_code}\n'
# 201

# ---- 3. wait until BOTH are Ready, four polls in a row ----
# One Ready reading is not enough: an agent whose MCP is not serving yet
exits and
# CrashLoopBackOffs, so readiness flaps. A run accepted mid-flap records 0
tasks.
STABLE=0; until [ "$STABLE" -ge 4 ]; do
  R=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/status?$SCOPE" -H
"Authorization: Bearer $TOKEN" \
      | python3 -c 'import sys,json; d=json.load(sys.stdin);
print(int(bool(d["tool_ready"] and d["agent_ready"])))')
  [ "$R" = 1 ] && STABLE=$((STABLE+1)) || STABLE=0
  echo "ready=$R stable=$STABLE"; sleep 10
```

```
done
# On OpenShift the Route 502s for ~10s AFTER the Service reports Ready, so
gate on the card too:
AGENT=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/status?$SCOPE" -H
"Authorization: Bearer $TOKEN" \
        | python3 -c 'import sys,json; print(json.load(sys.stdin)
["agent_name"])')
until curl -sf -o /dev/null "https://$AGENT-
team1.apps.ykt2.hcp.res.ibm.com/.well-known/agent-card.json"; do
  echo "waiting for the agent card"; sleep 5
done; sleep 15

# ---- 4. submit the run ----
export RUN=$(curl -s $CURL_OPTS -X POST "$SVC/benchmarks/$BENCH/runs" \
  -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"agent":"tool_calling","namespace":"team1","experiment":"default",
       "max_tasks":1,"max_parallel_sessions":1,"timeout_seconds":120}' \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)["run_id"])')
echo "run_id=$RUN"                                      # e.g. 20260902215628-
79e0713a

# ---- 5. poll to a terminal status, then print the verdict ----
while :; do
  S=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H
"Authorization: Bearer $TOKEN" \
      | python3 -c 'import sys,json; print(json.load(sys.stdin)["status"])')
  echo "status=$S"; case "$S" in succeeded|failed|error|cancelled) break ;;
esac; sleep 10
done
#   -> succeeded, summary
{"total":1,"succeeded":1,"evaluated_pass":1,"pass_rate":1.0,"wall_seconds":5.
4}
#      Note ~100 spans for ONE gsm8k task: ~91 are A2A/HTTP framework
internals.

# ---- 6. artifacts: list the S3 URLs, then mirror them locally ----
export PREFIX=$(curl -s $CURL_OPTS "$SVC/benchmarks/$BENCH/runs/$RUN" -H
"Authorization: Bearer $TOKEN" \
  | python3 -c 'import sys,json; print(json.load(sys.stdin)
["artifacts_prefix"])')
echo "$PREFIX"
#   -> ykt3-to-ykt2/benchmarker/keycloak-keycloak.apps.ykt2.hcp.res.ibm.com-
realms-rossoctl/gsm8k/20260902215628-79e0713a
mkdir -p "/tmp/autobench/$PREFIX" && (cd "/tmp/autobench/$PREFIX" && \
  for f in run.json report.ndjson report.parquet token_report.ndjson
token_report.parquet \
           span_report.ndjson span_report.parquet manifest.json; do
    curl -sS -O
"https://rossoctl-benchmarking.s3.us-east-1.amazonaws.com/$PREFIX/$f"; done
&& ls -la)

# ---- 7. analyse: per-task tokens, and whether the telemetry is trustworthy
----
cd "/tmp/autobench/$PREFIX"
```

```
python3 - <<'EOF'
import json
rows = [json.loads(l) for l in open("report.ndjson") if l.strip()]
print("%-6s %-38s %7s %7s %5s %6s  %s" % ("task", "model", "in", "out",
"llm", "tool", "passed"))
for r in sorted(rows, key=lambda x: str(x["task_id"])):
    print("%-6s %-38s %7d %7d %5s %6s  %s" % (
        r["task_id"], r["model"], r["llm_input_tokens"],
r["llm_output_tokens"],
        r["llm_count"], r["tool_count"], r["evaluation_result"]))
# llm<=1 alongside tool>=2 is impossible -- every tool call needs a preceding
model turn -- so it
# means the usage-bearing span was lost and the token counts understate the
real cost.
bad = [r for r in rows if (r["llm_count"] or 0) <= 1 and (r["tool_count"] or
0) >= 2]
print("lost token attribution:", len(bad), "of", len(rows))
EOF
#   -> 0      openai/Azure/gpt-5-mini-2025-08-07         320     471     2
1  True
#      lost token attribution: 0 of 1

# Which spans did the task actually invoke? A healthy task shows TWO chat
spans -- the
# max_tokens=1 capability probe plus the real call. One means the real call's
span was lost.
python3 - <<'EOF'
import collections, json
rows = [json.loads(l) for l in open("span_report.ndjson") if l.strip()]
print("spans:", len(rows))
for kind, n in collections.Counter(r["kind"] for r in rows).most_common():
    print("  %4d  %s" % (n, kind))
for r in rows:
    if r["kind"] == "chat":
        print("  chat  max_tokens=%s  in=%s  out=%s" % (
            r["request_max_tokens"], r["input_tokens"], r["output_tokens"]))
EOF
#   -> spans: 100  (other 91, phase 3, chat 2, tool 2, root 1, agent 1)
#      chat  max_tokens=1  in=None  out=None     <- the probe (rejected by a
reasoning model)
#      chat  max_tokens=None  in=320  out=471    <- the real call

# ---- 8. tear down ----
curl -s $CURL_OPTS -X DELETE "$SVC/benchmarks/$BENCH/deploy?$SCOPE" \
  -H "Authorization: Bearer $TOKEN" -o /dev/null -w 'teardown -> %{http_code}
\n'   # 204
```

6.1 The same thing in one command (`autobench-cli`)

`autobench-cli` is a stdlib-only client that performs §6.0 end to end — token, pre-clean, deploy, the readiness-stability and agent-card gates, run, 424 retry, poll, artifact listing and local mirror — and exits non-zero if the run did not succeed.

```
export BM_BASE="https://autobench-rossoctl-system.apps.ykt3.hcp.res.ibm.com"
```

```
export BM_ISS="https://keycloak-keycloak.apps.ykt2.hcp.res.ibm.com/realms/
rossoctl"
export BM_PASSWORD_FILE="$HOME/.rossoctl-ykt3/benchmarker.pass"     # chmod
600
export BM_INSECURE=1
export BM_CARD_TEMPLATE="https://{service}-
{namespace}.apps.ykt2.hcp.res.ibm.com/.well-known/agent-card.json"

# See "What you need on the client side" above for install options; the
shortest is:
#   uv pip install "rossoctl-autobench @
git+https://github.com/rossoctl/autobench" --no-deps
autobench-cli all --benchmark gsm8k --tasks 1 --timeout 120 --mirror
/tmp/autobench
```

Which prints, for the run above:

```
[17:56:12]   agent card 200 (https://exgentic-a2a-tool-calling-gsm8k-
team1.apps.ykt2...)
[17:56:28] run_id=20260902215628-79e0713a
[17:56:38] terminal=succeeded summary={"total": 1, ..., "pass_rate": 1.0,
"wall_seconds": 5.4}
[17:56:38]
artifacts_prefix=ykt3-to-ykt2/benchmarker/keycloak-...-rossoctl/gsm8k/
20260902215628-79e0713a
  json           468  https://rossoctl-benchmarking.s3.us-
east-1.amazonaws.com/.../run.json
  ...
[17:56:39] mirrored 8/8 -> /tmp/autobench/ykt3-to-ykt2/.../20260902215628-
79e0713a
  -rw-r--r--      3002  Sep 02 17:56  manifest.json
  -rw-r--r--      1035  Sep 02 17:56  report.ndjson
  -rw-r--r--     11984  Sep 02 17:56  report.parquet
  -rw-r--r--       468  Sep 02 17:56  run.json
  -rw-r--r--     52578  Sep 02 17:56  span_report.ndjson
  -rw-r--r--     10878  Sep 02 17:56  span_report.parquet
  -rw-r--r--       371  Sep 02 17:56  token_report.ndjson
  -rw-r--r--      4425  Sep 02 17:56  token_report.parquet
  8 files, 84,741 bytes

  cd /tmp/autobench/ykt3-to-ykt2/.../20260902215628-79e0713a

  status      succeeded
  pass_rate   1.0   (1/1)
  wall        5s
```

The trailing `cd` is there because the mirror path is absolute and deeply nested — retyping it relative to your current directory is the easy mistake. The sizes are also the quickest check that nothing arrived truncated. Feed that directory straight into §6.0 step 7 to analyse the run.

Each step is also a subcommand, so the CLI doubles as a way to see one HTTP call at a time:

```
autobench-cli whoami                                    # GET /hello
autobench-cli list                                      # GET /benchmarks
autobench-cli deploy    --benchmark gsm8k
autobench-cli wait      --benchmark gsm8k               # stability + card
```

```
gates
autobench-cli run       --benchmark gsm8k --tasks 1    # prints the run_id
autobench-cli poll      --benchmark gsm8k --run "$RUN"
autobench-cli report    --benchmark gsm8k --run "$RUN" # GET …/report (needs
MLflow)
autobench-cli artifacts --benchmark gsm8k --run "$RUN" --mirror
/tmp/autobench  # + ls -l
autobench-cli teardown  --benchmark gsm8k
```

No install? It runs straight from a checkout too — `python -m autobench.cli all …` (the module imports nothing beyond the standard library, so the server's dependencies are not needed to drive the API).

Useful options: `--parallel` (`max_parallel_sessions`), `--task-timeout`, `--model` for a deploy-time swap, `--preset auth-only|ibac-only|full` and repeatable `--plugin ibac:observe` for AuthBridge, `--no-deploy` to reuse a deployment, `--teardown` to clean up afterwards. Switching to KinD needs only three lines:

```
export BM_BASE="http://autobench.localtest.me:8080"
export BM_ISS="http://keycloak.localtest.me:8080/realms/rossoctl"
export BM_PASSWORD_FILE="$HOME/.rossoctl-kind/benchmarker.pass"; unset
BM_INSECURE BM_CARD_TEMPLATE
```

> For the full 12-run matrix use `reference/run-12.py` instead — see §6.3. It implements the same gates independently; consolidating both behind `autobench.cli` is a known follow-up.

6.2 Other validated flows

These map to the canonical parameterized runs and were validated on `ykt3` with S3 export.

```
# Run #2 — gsm8k, 10 tasks (validated: 9/10 pass)
curl -s -X POST "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer
$TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 10, "max_parallel_sessions": 1, "timeout_seconds": 600}'

# Run #3 — gsm8k, 50 tasks, 4-way concurrency (validated: 42/50, ~190s)
curl -s -X POST "$SVC/benchmarks/gsm8k/runs" -H "Authorization: Bearer
$TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 50, "max_parallel_sessions": 4, "timeout_seconds": 900}'

# Run #9 — tau2, 10 tasks (multi-turn; deploy tau2 first)
curl -s -X POST "$SVC/benchmarks/tau2/deploy" -H "Authorization: Bearer
$TOKEN" \
  -H "Content-Type: application/json" -d '{"namespace": "team1"}'
curl -s -X POST "$SVC/benchmarks/tau2/runs" -H "Authorization: Bearer $TOKEN"
\
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 10, "max_parallel_sessions": 1, "timeout_seconds": 1200}'

# Run #10 — tau2, 20 tasks, 4-way concurrency (validated: 17/20, ~915s —
```

```
needs raised timeout)
curl -s -X POST "$SVC/benchmarks/tau2/runs" -H "Authorization: Bearer $TOKEN"
\
  -H "Content-Type: application/json" \
  -d '{"max_tasks": 20, "max_parallel_sessions": 4, "timeout_seconds": 1800}'
```

6.3 The whole 12-run matrix in one command (`reference/run-12.py`)

Same idea as §6.1, one level up: instead of a single benchmark run, this drives all **12 canonical parameterized runs** — gsm8k at three sizes, a model swap, the four AuthBridge presets, tau2 at two sizes, appworld at two — deploying each leg fresh, mirroring every artifact, and writing one state file the report generators read.

Unlike `autobench-cli`, this one **is** repo-only: it is a matrix driver tied to `run12_specs.json`, not something you install. Clone first.

```
git clone https://github.com/rossoctl/autobench && cd autobench

# Same five variables as §6.1 — plus a label, which names the state file and
the reports.
export BM_BASE=https://autobench-rossoctl-system.apps.ykt3.hcp.res.ibm.com
export BM_ISS=https://keycloak-keycloak.apps.ykt2.hcp.res.ibm.com/realms/
rossoctl
export BM_PASSWORD_FILE="$HOME/.rossoctl-ykt3/benchmarker.pass"
export BM_INSECURE=1
export BM_CARD_TEMPLATE="https://{service}-
{namespace}.apps.ykt2.hcp.res.ibm.com/.well-known/agent-card.json"
export BM_LABEL=ocp-v124

python3 reference/run-12.py            # ~1h55m; log it: ... 2>&1 | tee
/tmp/run12-ocp.log
```

It ends with a per-leg summary and the state file path:

```
[21:03:57] === done in 6884s; 12 runs -> /tmp/autobench/run12-ocp-v124.json
[21:03:57]   #1  gsm8k     succeeded      pass=1.0   tasks=1
[21:03:57]   #2  gsm8k     succeeded      pass=1.0   tasks=10
...
[21:03:57]   #12 appworld  succeeded      pass=0.0   tasks=20
```

Then turn that state file into the three documents (all numbers derived from the mirrored artifacts — nothing transcribed):

```
# 1. the 12-run report. <date> is the date the RUNS executed, from the modal
run_id prefix.
python3 reference/gen-12run-report.py /tmp/autobench/run12-ocp-v124.json
v1.24 \
  "OpenShift — Service on ykt3, workloads on ykt2" results/12run-report-
v1.24-20260901-ocp.md

# 2. OCP vs KinD, once both matrices exist
python3 reference/gen-12run-comparison.py \
  /tmp/autobench/run12-ocp-v124.json OCP \
  /tmp/autobench/run12-kind-v124.json KinD v1.24 \
```

```
  results/12run-comparison-v1.24-20260902.md

# 3. AuthBridge plugin overhead. The judge log is what tells it how many tool
calls were actually
#    authorized — without it, per-preset medians mix judged and unjudged
calls and mislead.
oc -n rossoctl-system logs deploy/ibac-judge --since=6h --timestamps \
  | grep -vi healthz | awk '{print $1}' > /tmp/judge-ocp.ts
PEER_JSON=/tmp/autobench/run12-kind-v124.json PEER_LABEL="single-node KinD" \
PEER_JUDGE_TS=/tmp/judge-kind.ts \
python3 reference/gen-plugin-overhead.py /tmp/autobench/run12-ocp-v124.json
v1.24 \
  "OpenShift — Service on ykt3, workloads on ykt2" \
  results/12run-plugin-overhead-v1.24-20260901-ocp.md /tmp/judge-ocp.ts
```

Switching to KinD is the same three-line change as §6.1 plus a new label:

```
export BM_BASE=http://autobench.localtest.me:8080
export BM_ISS=http://keycloak.localtest.me:8080/realms/rossoctl
export BM_PASSWORD_FILE="$HOME/.rossoctl-kind/benchmarker.pass"
unset BM_INSECURE BM_CARD_TEMPLATE
export BM_LABEL=kind-v124
python3 reference/run-12.py            # ~1h55m on a single node too
```

Run the two platforms sequentially, not in parallel — they share the external LLM gateway, so overlapping them makes the wall-time comparison meaningless.

Other knobs, all optional:

Variable	Default	Use
`BM_ONLY`	all 12	comma-separated leg numbers, e.g. `BM_ONLY=7` to re-run one leg, `BM_ONLY=1,2,3` for a smoke pass
`BM_STABLE_POLLS`	`4`	consecutive ready polls required before a run is submitted
`BM_SETTLE_PLAIN` / `BM_SETTLE_SIDECAR`	`15` / `45`	post-ready settle seconds; sidecar deploys need longer
`BM_USER` / `BM_CLIENT`	`benchmarker` / `rossoctl`	non-default realm identity
`BM_PASSWORD`	—	the password inline, instead of `BM_PASSWORD_FILE`. Prefer the file (`chmod 600`): an exported variable leaks into `env`, process listings and shell history

`BM_ONLY` is how you repair a matrix without redoing it: a leg that failed to deploy can be re-run on its own and merged into the state file, which is exactly what happened to leg #7 of the v1.24 OCP matrix (a transient `DELETE -> 500` left a stale tool behind and the deploy hit `409`).

---

7. Known limits & error codes

Situation	Code	What to do
Missing/invalid bearer	`401`	fetch a fresh token (§2)
`iss` not in any instance file	`403`	add/repair the instance config (§3.1)
`/config` as non-benchmarker	`403`	authenticate as the `benchmarker` user
Setting a workload cred via `/config`	`422`	provision it as a cluster Secret instead (§3.3)
`plugin_preset/plugins/on_error` without `authbridge_enabled=true`	`422`	set `authbridge_enabled=true` so the sidecar is injected for the pipeline to take effect
`plugin_config_file` on deploy	`422`	local-path input with no HTTP analog; use `plugin_preset/plugins/on_error` instead
Run before deploy	`409`	`POST …/deploy` first
Run while not Ready	`424`	provision the named Secret(s), wait for Ready (§5.2)
Report with MLflow unconfigured	`409`	set MLflow read creds via `PUT /config` (§3.4)
Upstream Rossoctl/MLflow failure	`502`	transient upstream issue; retry

AuthBridge plugin presets (runs 4–8 of the ibac comparison): layer-3 plugin composition and `on_error` selection require overlaying the per-agent `authbridge-config-<agent>` ConfigMap via kubectl — which the HTTP-only Service cannot do — so those are honestly rejected with `422`. The one enactable knob is `authbridge_enabled=true` (injects the sidecar with the cluster-default pipeline).

appworld: registers and deploys correctly via the Service, but its MCP image hangs on its own per-task Venv-service health timeout (120s), independent of the Service. Treat appworld as externally blocked until that image is fixed.

---

8. Extending the catalog (adding or changing a benchmark)

The benchmark catalog is **code, not runtime data** — a static `BENCHMARKS` dict in `src/autobench/benchmarks/registry.py`. There is no database, config file, or admin API behind it, and the HTTP surface over the catalog is **read-only** (`GET /benchmarks`, `GET /benchmarks/{name}`). Adding or changing a benchmark is therefore a **source change + tests + image rebuild + redeploy**, deliberately: the definition pins container images, dataset env, and resource limits that must move in lockstep with the workload images, so every change is a

reviewable, git-tracked, image-versioned artifact rather than mutable state that could drift per instance.

> See the co-located contributor note `src/autobench/benchmarks/README.md` for the field-by-field reference and the per-benchmark env gotchas.

8.1 What a definition holds

Each entry is a `BenchmarkDefinition`: `name`, `mcp_image` (+ `mcp_image_tag`/`mcp_port`/ `mcp_path`), `tool_env`, `tool_resources`, `default_model`, `user_simulator` (multi-turn flag), and an `agents` map of `BenchmarkAgentSpec` (per-agent `container_image` + `extra_env` + `resources`). Secret references are declared with the `_secret_env(env, secret, key)` helper; `required_secrets()` derives the actionable `424` precheck message from them automatically.

8.2 Add a new benchmark

1. **Add a** `BenchmarkDefinition` **entry** to `BENCHMARKS` in `registry.py`:
   - `mcp_image` (+ tag/port/path) — the MCP tool image.
   - `tool_env` — `BENCHMARK_NAME` and any secret refs via `_secret_env`. The LLM base is **not** baked in: it comes from the instance's required `workload_llm.api_base` (KinD must use ETE's internal …`vpc-int`… endpoint; a deploy with no gateway configured is rejected with 422).
   - `agents={...}` — one `BenchmarkAgentSpec` per flavor. The `tool_calling` A2A image is shared across all current benchmarks; usually you reuse it and only the name gets a `-<benchmark>` suffix.
   - `user_simulator=True` **only** for multi-turn benchmarks — this is what makes `build_tool_request` inject `EXGENTIC_SET_BENCHMARK_USER_SIMULATOR_MODEL` into the MCP pod so the server-side simulator shares the run's model.
   - Any benchmark-quirk env — mind the documented gotchas: gsm8k needs `EXGENTIC_SET_BENCHMARK_RUNNER=direct`; tau2 needs `EXGENTIC_SET_BENCHMARK_ACTION_TIMEOUT=1000`; appworld **rejects** that same action-timeout override and crashes at startup if it's present.
2. **Add tests** in `tests/test_benchmarks.py` — assert `build_tool_request` / `build_agent_request` emit the expected images, env, secrets, and resources, mirroring the existing patterns.
3. **Rebuild + bump the image tag**, redeploy the Service. The new benchmark then appears in `GET /benchmarks` and is deployable/runnable at `/benchmarks/<newname>/`… with **no client change** — the request bodies are benchmark-agnostic (see §5).

8.3 Change an existing benchmark

Same mechanism — edit the entry (bump `mcp_image_tag`, change `default_model`, add an env var, adjust `tool_resources`), update tests, rebuild, redeploy. Because `required_secrets()` is derived from `tool_env` + the chosen agent's `extra_env`, changing a secret reference automatically updates the `424` "this benchmark requires secret(s): …" precheck message — no separate wiring.

8.4 What stays runtime-mutable (for contrast)

Per-instance state is *not* in the catalog and does not require a rebuild: instance config (files under `settings.instances_dir`, keyed by `iss`) and the benchmarker-only `PUT /config` overrides (MLflow read creds + S3). The benchmark catalog is intentionally the immutable, version-pinned part.